

Official Showcase.

QueryFast: The 2.5KB Powerhouse
for Reactive UI.

Reactive Todo Engine

Pure reactivity. No virtual DOM bottlenecks. Just lightning-fast updates.

```
const state = Q.state({ list: ['Task 1'], input: '' });

Q.component('Todo', () => {
  const root = Q.add('div.todo-app');
  Q.add('input.todo-input')
    .on('input', el => state.value.input = el.value)
    .appendTo(root);
  Q.add('button').content('Add').on('click', () => {
    state.value = { list: [...state.value.list, state.value.input]
  }).appendTo(root);

  Q.add('div').bind(state, s => s.list.map(t =>
    `<div class="todo-item">${t}</div>`
  )).join(''), 'html').appendTo(root);
  return root;
});
```

Component Factory

Compose complex UIs with simple functions and custom props.

```
Q.component('Card', (props = {}) => {  
  const { title = 'Default', color = '#333' } = props;  
  return Q.add('div')  
    .ui({ bg: color, padding: '30px', borderRadius: '10px', color })  
    .html(`<h4>${title}</h4>`);  
});  
  
Q.mount('Card', '#demo', { title: 'Premium', color: '#7928ca' });
```

Learning the Core

Philosophy

QueryFast is built on the principle of ****Direct DOM Reactivity****. We don't hide the DOM from you; we give the DOM "nerves" to react to state changes automatically.

Lifecycle

1. **Define State:** `const s = Q.state(val)`
2. **Create UI:** `const el = Q.add('div')`
3. **Bind:** `el.bind(s, mapFn, 'text')`

Official API Manual

Comprehensive documentation for every method in the QueryFast ecosystem.

1. Global Functions (Q)

Q(selector)

Args: string | Node | Node[] | QCollection

Returns: QCollection

The universal selector. Wraps results in a chainable array-like object.

Q.add(selector)

Args: string (e.g., 'div#id.class')

Returns: QCollection (1 element)

Creates a new element. Parses tag, ID, and classes from the string.

Q.state(initial)

Args: any

Returns: Proxy

Creates a reactive state. Use `state.value` to read/write.
Use `state._subscribe(fn)` for manual observation.

Q.component(name, factory)

Args: string, (props) => QCollection

Registers a reusable UI component.

Q.use(name, props?)

Returns: QCollection

Creates an instance of a registered component.

Q.mount(name, target, props?)

Returns: QCollection

Instantiates and appends a component to the DOM.

2. QCollection Methods

.appendTo(target)

Args: string | Node | QCollection

Appends all elements in the collection to the target.

.content(value?)

Args: string (optional)

Getter/Setter for textContent. If multiple elements, getter returns the first.

.html(value?)

Getter/Setter for innerHTML.

.attr(name, value?)

Value: string | null | false | undefined

If value is null/false/undefined, the attribute is removed.

.on(type, handler)

Handler: (el, event, meta?) => void

Supports 'hover' special event. Handler for hover: (el, isActive, event).

.ui(styleObj)

Shortcuts: bg (background), text (color).

Directly sets CSS properties on the style object.

.bind(state, mapFn, mode)

Modes: text, html, attr, class, css

The bridge between state and DOM.